

Reto | Mercadotecnia telefónica con aprendizaje supervisado

Guillermo Ortega Arzate

✓ Introducción

Los programas de telemarketing se han venido usando por las empresas durante años como una forma de comunicarse con clientes potenciales de manera directa por vía telefónica. En los últimos años el uso de las técnicas de inteligencia artificial ha permitido potencializar el impacto de estos programas de mercadotecnia.

Un banco ha lanzado **un nuevo plan de inversión** a largo plazo a través de un programa de telemarketing y desea saber que tan exitoso se puede considerar este programa utilizando técnicas de aprendizaje supervisado. El programa es lanzado a clientes del banco, por lo que este estudio ayudará a optimizar los esfuerzos al conocer las características de los clientes más **propensos a adquirir dicho plan de inversión**.

En la presente actividad tu objetivo es predecir si el entrevistado adquirirá el plan de inversión bancario de acuerdo con varias características que se conocen de dicho cliente.

This dataset is public available for research. The details are described in [Moro et al., 2011]. Please include this citation if you plan to use this database:

[Moro et al., 2011] S. Moro, R. Laureano and P. Cortez. Using Data Mining for Bank Direct Marketing: An Application of the CRISP-DM Methodology. In P. Novais et al. (Eds.), Proceedings of the European Simulation and Modelling Conference - ESM'2011, pp. 117-121, Guimarães, Portugal, October, 2011. EUROSIS.

Available at: [pdf] <http://hdl.handle.net/1822/14838> [bib] <http://www3.dsi.uminho.pt/pcortez/bib/2011-esm-1.txt>

```
1 from sklearn.preprocessing import OrdinalEncoder, LabelEncoder, StandardScaler
2 from sklearn.model_selection import train_test_split
3 from sklearn.neural_network import MLPClassifier
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
6                             confusion_matrix, roc_auc_score, roc_curve)
7 import warnings
8 import numpy as np
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 import os
13
14 warnings.filterwarnings('ignore')
15
16 # Configuración de visualización
17 plt.rcParams['figure.figsize'] = (12, 6)
18 plt.rcParams['font.size'] = 11
19 sns.set_style('whitegrid')
```

```
20
21 # =====
22 # 4. CARGA DE DATOS
23 # =====
24 print("\n" + "=" * 70)
25 print("CARGA DE DATOS")
26 print("=" * 70)
27
28 if not os.path.exists('bank_marketing_RET0_DS_AS.csv'):
29     raise SystemExit("El archivo 'bank_marketing_RET0_DS_AS.csv' no existe")
30
31 data = pd.read_csv('bank_marketing_RET0_DS_AS.csv')
32
33 print(f"Dataset cargado: {data.shape[0]} filas, {data.shape[1]} columnas")
34 print(f"\nPrimeras filas:")
35 print(data.head())
36
37 # =====
38 # 5. ANÁLISIS EXPLORATORIO
39 # =====
40 print("\n" + "=" * 70)
41 print("ANÁLISIS EXPLORATORIO")
42 print("=" * 70)
43
44 print(f"\nTipos de datos:")
45 print(data.dtypes)
46
47 print(f"\nValores nulos:")
48 print(data.isnull().sum())
49
50 print(f"\nVariable objetivo (y):")
51 print(data['y'].value_counts())
52 # determinar porcentaje observado, nuestro modelo deberá ser mejor que esto
53 porcentajeObservado = data['y'].value_counts(normalize=True)[1] * 100
54 print(f" Porcentaje observado: {porcentajeObservado:.2f}%")
55
56 numericas_cols = data.select_dtypes(include=['int64', 'float64']).columns.tolist()
57 categoricas_cols = data.select_dtypes(include='object').columns.tolist()
58
59 print("\n" + "=" * 70)
60 print("Variables categóricas - valores únicos")
61 print("=" * 70)
62 for col in categoricas_cols:
63     print(f"\n{col}: {data[col].nunique()} valores únicos")
64     print(data[col].value_counts())
65
66 # =====
67 # 6.7.8 PREPROCESAMIENTO, TRANSFORMACIÓN
68 # =====
69 print("\n" + "=" * 70)
70 print("PREPROCESAMIENTO")
71 print("=" * 70)
72
73 # Conforme a los datos NO hay datos sin imputar
74 # Imputación
75 for col in numericas_cols:
76     if data[col].isnull().sum() > 0:
77         mediana = data[col].median()
78         data[col].fillna(mediana, inplace=True)
```

```

79
80 for col in categoricas_cols:
81     if data[col].isnull().sum() > 0:
82         moda = data[col].mode()[0]
83         data[col].fillna(modas, inplace=True)
84
85 print("No hay valores faltantes a imputar")
86
87 # Codificación
88 X_encoded = data.copy()
89
90
91 ## se estuvieron haciendo pruebas con diferentes codificadores
92 ## la primera prueba fue con OneHotEncoder y se generaban demasiadas variables
93 ## la segunda prueba fue usando OrdinalEncoder con todas las categóricas pero
94 ## desde el punto de vista teórico el Ordinal es cuando las variables tiene un orden
95 ## y no es el caso de nuestros datos por lo que usa:
96 ## Variables ordinales → OrdinalEncoder
97 ## Variables nominales → LabelEncoder
98
99 # Conforme a la exploración se tiene que:
100 # las columnas 'default', 'housing', 'loan' son binarias (LabelEncoder):
101 print("\nCodificación de variables binarias: default, housing, loan.")
102
103 # las columnas job, marital, contact, poutcome son categóricas sin que sus valores reflejen un orden (LabelEncoder)
104 print("\nCodificación de variables categóricas sin orden: job, marital, contact, poutcome.")
105
106 # las columnas education, month son categóricas con orden (OrdinalEncoder)
107 # no es que un mes sea más relevante que otro pero los meses tienen un orden secuencial
108 print("\nCodificación de variables categóricas con orden: education, month")
109
110 # las variables age, day, duration, campaign, pdays, previous son numéricas discretas
111 print("\nVariables numéricas discretas: age, day, duration, campaign, pdays, previous")
112
113 # La variable balance es numérica continua
114 print("\nVariable numérica continua: balance")
115
116 # codificar las binarias y categóricas sin orden
117 col_labelEncoder = ['job', 'marital', 'default', 'housing', 'loan', 'contact', 'poutcome', 'y']
118 le = LabelEncoder()
119 for col in col_labelEncoder:
120     X_encoded[col] = le.fit_transform(X_encoded[col])
121
122 # Mes como ordinal
123 # no es que un mes sea más relevante que otro pero los meses tienen un orden secuencial
124 mes_orden = ['jan', 'feb', 'mar', 'apr', 'may', 'jun', 'jul', 'aug', 'sep', 'oct', 'nov', 'dec']
125 oe_comp = OrdinalEncoder(categories=[mes_orden])
126 X_encoded['month'] = oe_comp.fit_transform(X_encoded[['month']]).astype(int)
127
128 # educacion con ordinal
129 education_orden = ['unknown', 'primary', 'secondary', 'tertiary']
130 oe_comp = OrdinalEncoder(categories=[education_orden])
131 X_encoded['education'] = oe_comp.fit_transform(X_encoded[['education']]).astype(int)
132
133
134 # pdays tiene valores de -1 y tiene sesgo, vamos a reemplazar los valores de -1 por 0
135 X_encoded['pdays'] = X_encoded['pdays'].replace(-1, 0)
136
137 # Balance tiene valores negativos y tiene sesgo, vamos a reemplazar los valores negativos por 0

```

```

137 # balance tiene valores negativos y tiene sesgo, vamos a reemplazar los valores negativos con 0
138 X_encoded['balance'] = X_encoded['balance'].apply(lambda x: 0 if x < 0 else x)
139
140 # Transformaciones de sesgo
141 highly_skewed_features = ['balance', 'pdays', 'previous', 'campaign', 'duration', 'age']
142 X_transformed = X_encoded.copy()
143 for col in highly_skewed_features:
144     X_transformed[col] = np.log1p(X_transformed[col])
145
146 X_encoded = X_transformed.copy()
147 print("Transformaciones aplicadas")
148
149 X = X_encoded.drop('y', axis=1)
150 y = X_encoded['y']
151
152 # =====
153 # 9. PARTICIÓN DE DATOS
154 # =====
155 print("\n" + "=" * 70)
156 print("PARTICIÓN DE DATOS")
157 print("=" * 70)
158
159 # División clara: 60% entrenamiento, 20% validación, 20% prueba
160 X_train, X_temp, y_train, y_temp = train_test_split(
161     X, y, test_size=0.4, random_state=42)
162
163 X_validation, X_test, y_validation, y_test = train_test_split(
164     X_temp, y_temp, test_size=0.5, random_state=42)
165
166 print(f"Entrenamiento: {len(X_train):5d} ({len(X_train)/len(X)*100:.1f}%")
167 print(f"Validación: {len(X_validation):5d} ({len(X_validation)/len(X)*100:.1f}%")
168 print(f"Prueba: {len(X_test):5d} ({len(X_test)/len(X)*100:.1f}%")
169
170 # =====
171 # NORMALIZACIÓN
172 # =====
173 print("\n" + "=" * 70)
174 print("ESCALADO DE CARACTERÍSTICAS")
175 print("=" * 70)
176
177 scaler = StandardScaler()
178 X_train_scaled = scaler.fit_transform(X_train)
179 X_validation_scaled = scaler.transform(X_validation)
180 X_test_scaled = scaler.transform(X_test)
181
182 print("✅ Características escaladas con StandardScaler")
183 print(f" Media en entrenamiento (primeras 3): {X_train_scaled.mean(axis=0)[:3]}")
184 print(f" Desv. Est. en entrenamiento (primeras 3): {X_train_scaled.std(axis=0)[:3]}")
185
186 # =====
187 # 10. REGRESIÓN LOGÍSTICA
188 # =====
189 print("\n" + "=" * 70)
190 print("REGRESIÓN LOGÍSTICA")
191 print("=" * 70)
192
193
194 # Durante las pruebas se notó que el método newton-cg era muy tardado (más de 5 minutos
195 # en colab ambiente default y 10000 iteraciones).

```

```

196 # Se revisó la literatura y para el tamaño del dataset confirmaba lo que observamos
197 # y se recomendaba usar sólo lbfgs y liblinear en consideración de lo anterior y que los otros métodos
198 # tiene muy poco impacto en la exactitud. Tanto sag como saga están diseñados
199 # para datasets 100x nuestro tamaño.
200 parameters = {
201     'C': [0.001, 0.01, 0.1, 1.0, 10.0],
202     'penalty': ['l2'],
203     'solver': ['lbfgs', 'liblinear']
204 }
205
206 best_rl_score = 0
207 best_rl_params = {}
208 best_rl_model = None
209
210 print("\nBúsqueda de parámetros usando CONJUNTO DE VALIDACIÓN:")
211 print(f"{'C':>10} {'Penalty':>10} {'Solver':>15} {'Val Accuracy':>15}")
212 print("-" * 52)
213
214 for C in parameters['C']:
215     for penalty in parameters['penalty']:
216         for solver in parameters['solver']:
217             try:
218                 model = LogisticRegression(
219                     C=C,
220                     penalty=penalty,
221                     solver=solver,
222                     max_iter=500,
223                     random_state=42
224                 )
225                 model.fit(X_train_scaled, y_train)
226
227                 val_pred = model.predict(X_validation_scaled)
228                 val_acc = accuracy_score(y_validation, val_pred)
229
230                 print(f"{'C':10.3f} {'penalty':>10} {'solver':>15} {'val_acc':>15.4f}")
231
232                 if val_acc > best_rl_score:
233                     best_rl_score = val_acc
234                     best_rl_params = {'C': C, 'penalty': penalty, 'solver': solver}
235                     best_rl_model = model
236
237             except Exception as e:
238                 pass
239
240 print(f"\nMejores parámetros encontrados:")
241 print(f"  C: {best_rl_params['C']}")
242 print(f"  Penalty: {best_rl_params['penalty']}")
243 print(f"  Solver: {best_rl_params['solver']}")
244 print(f"  Score en validación: {best_rl_score:.4f}")
245
246 # Evaluación detallada en validación
247 y_pred_rl_val = best_rl_model.predict(X_validation_scaled)
248 y_pred_proba_rl_val = best_rl_model.predict_proba(X_validation_scaled)[:, 1]
249
250 print("\n📊 MÉTRICAS EN CONJUNTO DE VALIDACIÓN:")
251 print(f"  Accuracy: {accuracy_score(y_validation, y_pred_rl_val):.4f} (% de predicciones correctas)")
252 print(f"  Precision: {precision_score(y_validation, y_pred_rl_val):.4f} (de los 'Sí' predichos, cuántos son correctos)")
253 print(f"  Recall: {recall_score(y_validation, y_pred_rl_val):.4f} (de los 'Sí' reales, cuántos detectamos)")
254 print(f"  F1 Score: {f1_score(y_validation, y_pred_rl_val):.4f} (balance entre precisión y recall)")

```

```

254 print(f" F1-Score: {f1_score(y_validation, y_pred_rl_val):.4f} (balance entre precision y recall) ")
255 print(f" ROC-AUC: {roc_auc_score(y_validation, y_pred_proba_rl_val):.4f} (probabilidad de acertar a un Sí o un No)")
256
257 cm_rl_val = confusion_matrix(y_validation, y_pred_rl_val)
258 print(f"\n Matriz de confusión (validación):")
259 print(cm_rl_val)
260
261 # =====
262 # 11. RED NEURONAL
263 # =====
264 print("\n" + "=" * 70)
265 print("RED NEURONAL")
266 print("=" * 70)
267
268 # al principio se probó con aquitecturas de redes de dos capas ocultas del mismo
269 # tamaño, pero la cantidad de neuronas variaba mucho por ejemplo entre 16 y 26
270 # dependiendo de la semilla que seleccionara.
271 # se revisó el siguiente documento https://www.heatonresearch.com/2017/06/01/hidden-layers.html
272 # y conforme a sus recomendaciones se probó con las siguientes arquitecturas.
273 # El 16 sale de que debe ser menor de la cantidad de entradas (17) + salida
274 # no se recomienda más de dos capas para la complejidad del problema.
275 # La primera capa aprende características, segunda capa patrones
276 hidden_layers_to_try = [(16,), (16,16), (32,16), (64,32), (128,64)]
277 best_nn_score = 0
278 best_nn_params = {}
279 best_nn_model = None
280
281 print("\nBúsqueda de parámetros usando CONJUNTO DE VALIDACIÓN:")
282 print(f"{'Hidden Layers':>30} {'Val Accuracy':>15} {'Iteraciones':>15}")
283 print("-" * 62)
284
285 for hidden_layers in hidden_layers_to_try:
286     try:
287         model = MLPClassifier(
288             hidden_layer_sizes=hidden_layers,
289             max_iter=500, # Reducido para evitar sobreajuste
290             alpha=0.0001, # Regularización L2 (valor default)
291             batch_size=32, # default es 100 pero literatura indica empezar en 32, 64
292             learning_rate_init=0.001, # recomendado por la literatura
293             early_stopping=True, # detenerse cuando no mejora validación
294             validation_fraction=0.2, # usar 20% del entrenamiento para validar (cuando early_stopping = True)
295             n_iter_no_change=20, # previene sobreajuste, iteraciones máximas sin superar tolerancia a mejora
296             random_state=42, # semilla para reproducir los cálculos, 42 es usado en los ejemplos
297             verbose=0 # no mandar actualizaciones a la consola (default)
298         )
299
300         model.fit(X_train_scaled, y_train)
301
302         val_pred = model.predict(X_validation_scaled)
303         val_acc = accuracy_score(y_validation, val_pred)
304
305         print(f"{'Hidden Layers':>30} {'Val Acc':>15.4f} {'model.n_iter_':>15}")
306
307         if val_acc > best_nn_score:
308             best_nn_score = val_acc
309             best_nn_params = {'hidden_layers': hidden_layers}
310             best_nn_model = model
311
312     except Exception as e:

```

```

313     print(f"Error con {hidden_layers}: {e}")
314
315 print(f"\nMejores parámetros encontrados:")
316 print(f"  Hidden layers: {best_nn_params['hidden_layers']}")
317 print(f"  Score en validación: {best_nn_score:.4f}")
318
319 # Evaluación detallada en validación
320 y_pred_nn_val = best_nn_model.predict(X_validation_scaled)
321 y_pred_proba_nn_val = best_nn_model.predict_proba(X_validation_scaled)[:, 1]
322
323 print("\n📊 MÉTRICAS EN CONJUNTO DE VALIDACIÓN:")
324 print(f"  Accuracy:   {accuracy_score(y_validation, y_pred_nn_val):.4f} (% de predicciones correctas)")
325 print(f"  Precision:  {precision_score(y_validation, y_pred_nn_val):.4f} (de los 'Sí' predichos, cuántos son correctos)")
326 print(f"  Recall:     {recall_score(y_validation, y_pred_nn_val):.4f} (de los 'Sí' reales, cuántos detectamos)")
327 print(f"  F1-Score:   {f1_score(y_validation, y_pred_nn_val):.4f} (balance entre precision y recall)")
328 print(f"  ROC-AUC:    {roc_auc_score(y_validation, y_pred_proba_nn_val):.4f} (probabilidad de acertar a un Sí o un No)")
329
330 cm_nn_val = confusion_matrix(y_validation, y_pred_nn_val)
331 print(f"\n📄 Matriz de confusión (validación):")
332 print(cm_nn_val)
333
334 # =====
335 # 12. COMPARACIÓN Y SELECCIÓN DEL MEJOR MODELO
336 # =====
337 print("\n" + "=" * 70)
338 print("COMPARACIÓN DE MODELOS")
339 print("=" * 70)
340
341 print(f"\nAccuracy en VALIDACIÓN:")
342 print(f"  Regresión Logística: {best_rl_score:.4f}")
343 print(f"  Red Neuronal:        {best_nn_score:.4f}")
344
345 if best_rl_score > best_nn_score:
346     print("\n✅ SELECCIONADO: Regresión Logística")
347     best_model = best_rl_model
348     best_model_name = "Regresión Logística"
349     best_metrics_val = {
350         'accuracy': accuracy_score(y_validation, y_pred_rl_val),
351         'precision': precision_score(y_validation, y_pred_rl_val),
352         'recall': recall_score(y_validation, y_pred_rl_val),
353         'f1': f1_score(y_validation, y_pred_rl_val),
354         'roc_auc': roc_auc_score(y_validation, y_pred_proba_rl_val)
355     }
356 else:
357     print("\n✅ SELECCIONADO: Red Neuronal")
358     best_model = best_nn_model
359     best_model_name = "Red Neuronal"
360     best_metrics_val = {
361         'accuracy': accuracy_score(y_validation, y_pred_nn_val),
362         'precision': precision_score(y_validation, y_pred_nn_val),
363         'recall': recall_score(y_validation, y_pred_nn_val),
364         'f1': f1_score(y_validation, y_pred_nn_val),
365         'roc_auc': roc_auc_score(y_validation, y_pred_proba_nn_val)
366     }
367
368 # =====
369 # EVALUACIÓN FINAL EN CONJUNTO DE PRUEBA
370 # =====
371 print("\n" + "=" * 70)

```

```

372 print("EVALUACIÓN FINAL EN CONJUNTO DE PRUEBA")
373 print("=" * 70)
374
375 print(f"\nModelo seleccionado: {best_model_name}\n")
376
377 y_pred_test = best_model.predict(X_test_scaled)
378 y_pred_proba_test = best_model.predict_proba(X_test_scaled)[:, 1]
379
380 test_metrics = {
381     'accuracy': accuracy_score(y_test, y_pred_test),
382     'precision': precision_score(y_test, y_pred_test),
383     'recall': recall_score(y_test, y_pred_test),
384     'f1': f1_score(y_test, y_pred_test),
385     'roc_auc': roc_auc_score(y_test, y_pred_proba_test)
386 }
387
388 print("📊 MÉTRICAS FINALES EN CONJUNTO DE PRUEBA:")
389 print(f" Accuracy: {test_metrics['accuracy']:.4f} (% de predicciones correctas)")
390 print(f" Precision: {test_metrics['precision']:.4f} (de los 'Sí' predichos, cuántos son correctos)")
391 print(f" Recall: {test_metrics['recall']:.4f} (de los 'Sí' reales, cuántos detectamos)")
392 print(f" F1-Score: {test_metrics['f1']:.4f} (balance entre precision y recall)")
393 print(f" ROC-AUC: {test_metrics['roc_auc']:.4f} (probabilidad de acertar a un Sí o un No)")
394
395 cm_test = confusion_matrix(y_test, y_pred_test)
396 print(f"\n📄 Matriz de confusión (PRUEBA):")
397 print(cm_test)
398
399 # Análisis de matriz de confusión
400 tn, fp, fn, tp = cm_test.ravel()
401 print(f"\nDesglose:")
402 print(f" Verdaderos Positivos (TP): {tp}")
403 print(f" Verdaderos Negativos (TN): {tn}")
404 print(f" Falsos Positivos (FP): {fp}")
405 print(f" Falsos Negativos (FN): {fn}")
406
407 # =====
408 # VISUALIZACIONES
409 # =====
410 print("\n" + "=" * 70)
411 print("VISUALIZACIONES")
412 print("=" * 70)
413
414 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
415
416 # Matriz de confusión
417 ax = axes[0, 0]
418 sns.heatmap(cm_test, annot=True, fmt='d', cmap='Blues', ax=ax,
419             xticklabels=['No', 'Sí'], yticklabels=['No', 'Sí'])
420 ax.set_title(f'Matriz de Confusión - {best_model_name}\n(Conjunto de Prueba)', fontweight='bold')
421 ax.set_ylabel('Verdadero')
422 ax.set_xlabel('Predicho')
423
424 # Curva ROC
425 ax = axes[0, 1]
426 fpr, tpr, _ = roc_curve(y_test, y_pred_proba_test)
427 ax.plot(fpr, tpr, linewidth=2, label=f'ROC (AUC={test_metrics["roc_auc"]:.4f})', color='blue')
428 ax.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Clasificador Aleatorio')
429 ax.set_xlabel('Tasa de Falsos Positivos')

```



```

430 ax.set_ylabel('Tasa de Verdaderos Positivos')
431 ax.set_title('Curva ROC - Conjunto de Prueba', fontweight='bold')
432 ax.legend()
433 ax.grid(True, alpha=0.3)
434
435 # Comparación de métricas
436 ax = axes[1, 0]
437 metrics_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']
438 metrics_values = [test_metrics['accuracy'], test_metrics['precision'],
439                   test_metrics['recall'], test_metrics['f1'], test_metrics['roc_auc']]
440 bars = ax.bar(metrics_names, metrics_values, color=['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd'])
441 ax.set_ylabel('Puntuación')
442 ax.set_title('Métricas de Rendimiento - Conjunto de Prueba', fontweight='bold')
443 ax.set_ylim([0, 1])
444 ax.grid(True, alpha=0.3, axis='y')
445 # Añadir valores en las barras
446 for bar, value in zip(bars, metrics_values):
447     height = bar.get_height()
448     ax.text(bar.get_x() + bar.get_width()/2., height,
449            f'{value:.3f}', ha='center', va='bottom', fontweight='bold')
450
451 # Comparación validación vs prueba
452 ax = axes[1, 1]
453 x_pos = np.arange(len(metrics_names))
454 width = 0.35
455 val_values = [best_metrics_val['accuracy'], best_metrics_val['precision'],
456              best_metrics_val['recall'], best_metrics_val['f1'], best_metrics_val['roc_auc']]
457 ax.bar(x_pos - width/2, val_values, width, label='Validación', alpha=0.8)
458 ax.bar(x_pos + width/2, metrics_values, width, label='Prueba', alpha=0.8)
459 ax.set_ylabel('Puntuación')
460 ax.set_title('Comparación Validación vs Prueba', fontweight='bold')
461 ax.set_xticks(x_pos)
462 ax.set_xticklabels(metrics_names, rotation=45, ha='right')
463 ax.legend()
464 ax.set_ylim([0, 1])
465 ax.grid(True, alpha=0.3, axis='y')
466
467 # obtener la máxima variación entre pruebas y validación
468 max_diff = max(abs(np.array(val_values) - np.array(metrics_values)))
469
470 plt.tight_layout()
471 plt.savefig('resultados_finales.png',
472           dpi=150, bbox_inches='tight')
473 print("\n✅ Gráficas guardadas en 'resultados_finales.png'")
474 plt.show()
475
476 # =====
477 # 13. CONCLUSIONES
478 # =====
479 print("\n" + "=" * 70)
480 print("CONCLUSIONES")
481 print("=" * 70)
482
483 print(f"""
484 MODELO SELECCIONADO: {best_model_name}
485
486 - El dataset histórico indica que hay 42% de clientes que aceptarán la oferta.
487 - El modelo seleccionado predice con un Sí o un No en un porcentaje de {test_metrics['accuracy']*100:.4f}
488

```

```
489 RENDIMIENTO FINAL:
490 - Accuracy: {test_metrics['accuracy']:.4f} (% de predicciones correctas)
491 - Precision: {test_metrics['precision']:.4f} (de los "Sí" predichos, cuántos son correctos)
492 - Recall: {test_metrics['recall']:.4f} (de los "Sí" reales, cuántos detectamos)
493 - F1-Score: {test_metrics['f1']:.4f} (balance entre precision y recall)
494 - ROC-AUC: {test_metrics['roc_auc']:.4f} (probabilidad de acertar a un Sí o un No)
495
496 COMPARACIÓN CON VALIDACIÓN:
497 - No hay evidencia de SOBREAJUSTE (métricas validación ≈ métricas prueba)
498 La desviación máxima encontrada fue de {max_diff:.2f}
499 (ver gráfica de barras "Comparación Validación vs Pruebas")
500 - El modelo no está SUBENTRENADO, el porcentaje histórico de SI (aceptación) es de
501 {porcentajeObservado:.2f}% mientras que el modelo tiene una precisión de {test_metrics['precision']*100:.2f}%
502
503 INTERPRETACIÓN PARA MARKETING:
504 - Por cada 100 clientes predichos como "aceptarán la oferta",
505 {test_metrics['precision']*100:.0f} realmente lo harán.
506
507 - De los clientes que realmente aceptarían, el modelo detecta
508 el {test_metrics['recall']*100:.0f}% de ellos.
509
510 RECOMENDACIONES:
511 1. El modelo puede usarse para identificar clientes con mayor probabilidad
512 de aceptar ofertas de mercadotecnia telefónica.
513
514 2. La métrica de (('Recall' if test_metrics['recall'] < test_metrics['precision'] else 'Precision')) es más crítica
515 dependiendo del objetivo (detectar máximo de clientes vs. precisión máxima).
516
517 3. Se recomienda usar el ROC-AUC como métrica principal para
518 comparación con futuros modelos, puesto que representa
519 la probabilidad de que el modelo ordene correctamente
520 un cliente que dice "Sí" y otro que dice "No", elegidos al azar
521 "''")
522
```

